

Ultimate Array Notes in Java

1. What is Array?

Array is a linear data structure used to store multiple values of same datatype in contiguous memory locations.

Features:

- Fixed Size
 - Same Datatype
 - Index Based
 - Fast Access
-

2. Array Declaration

```
int[] arr;
```

3. Array Creation

```
int[] arr = new int[5];
```

4. Array Initialization

```
int[] arr = {10,20,30,40,50};
```

5. Access Elements

```
System.out.println(arr[0]);
```

6. Update Elements

```
arr[1] = 100;
```

7. Traversal using Loop

```
for(int i = 0; i < arr.length; i++) {  
    System.out.print(arr[i] + " ");  
}
```

8. Traversal using For Each Loop

```
for(int x : arr) {  
    System.out.print(x + " ");  
}
```

9. Find Length

```
System.out.println(arr.length);
```

10. Search Element (Linear Search)

```
boolean search(int[] arr, int key) {  
    for(int i = 0; i < arr.length; i++) {  
        if(arr[i] == key)  
            return true;  
    }  
}
```

```
        return false;
    }
```

11. Find Maximum Element

```
int max(int[] arr) {
    int max = arr[0];
    for(int i = 1; i < arr.length; i++) {
        if(arr[i] > max)
            max = arr[i];
    }
    return max;
}
```

12. Find Minimum Element

```
int min(int[] arr) {
    int min = arr[0];
    for(int i = 1; i < arr.length; i++) {
        if(arr[i] < min)
            min = arr[i];
    }
    return min;
}
```

13. Sum of Array

```
int sum(int[] arr) {  
  
    int sum = 0;  
  
    for(int i = 0; i < arr.length; i++) {  
  
        sum += arr[i];  
    }  
  
    return sum;  
}
```

14. Average of Array

```
double average(int[] arr) {  
  
    int sum = 0;  
  
    for(int i = 0; i < arr.length; i++) {  
  
        sum += arr[i];  
    }  
  
    return (double)sum / arr.length;  
}
```

15. Reverse Array

```
void reverse(int[] arr) {  
  
    int start = 0;  
    int end = arr.length - 1;  
  
    while(start < end) {  
  
        int temp = arr[start];
```

```
        arr[start] = arr[end];

        arr[end] = temp;

        start++;
        end--;
    }
}
```

16. Copy Array

```
int[] copy(int[] arr) {

    int[] copy = new int[arr.length];

    for(int i = 0; i < arr.length; i++) {

        copy[i] = arr[i];
    }

    return copy;
}
```

17. Sort Array

```
void sort(int[] arr) {

    java.util.Arrays.sort(arr);
}
```

18. Bubble Sort

```
void bubbleSort(int[] arr) {

    int n = arr.length;

    for(int i = 0; i < n - 1; i++) {
```

```

        for(int j = 0; j < n - i - 1; j++) {

            if(arr[j] > arr[j + 1]) {

                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

```

19. Selection Sort

```

void selectionSort(int[] arr) {

    int n = arr.length;

    for(int i = 0; i < n - 1; i++) {

        int min = i;

        for(int j = i + 1; j < n; j++) {

            if(arr[j] < arr[min])
                min = j;
        }

        int temp = arr[min];
        arr[min] = arr[i];
        arr[i] = temp;
    }
}

```

20. Insertion Sort

```

void insertionSort(int[] arr) {

    for(int i = 1; i < arr.length; i++) {

```

```
int key = arr[i];
int j = i - 1;

while(j >= 0 && arr[j] > key) {

    arr[j + 1] = arr[j];
    j--;
}

arr[j + 1] = key;
}
```

21. Binary Search

```
int binarySearch(int[] arr, int key) {

    int low = 0;
    int high = arr.length - 1;

    while(low <= high) {

        int mid = (low + high) / 2;

        if(arr[mid] == key)
            return mid;

        else if(arr[mid] < key)
            low = mid + 1;

        else
            high = mid - 1;
    }

    return -1;
}
```

22. Insert Element in Array

```
int[] insert(int[] arr, int pos, int val) {  
  
    int[] newArr = new int[arr.length + 1];  
  
    for(int i = 0; i < pos; i++)  
        newArr[i] = arr[i];  
  
    newArr[pos] = val;  
  
    for(int i = pos; i < arr.length; i++)  
        newArr[i + 1] = arr[i];  
  
    return newArr;  
}
```

23. Delete Element from Array

```
int[] delete(int[] arr, int pos) {  
  
    int[] newArr = new int[arr.length - 1];  
  
    for(int i = 0; i < pos; i++)  
        newArr[i] = arr[i];  
  
    for(int i = pos + 1; i < arr.length; i++)  
        newArr[i - 1] = arr[i];  
  
    return newArr;  
}
```

24. Frequency Count

```
void frequency(int[] arr) {  
  
    for(int i = 0; i < arr.length; i++) {  
  
        int count = 1;
```



```

        boolean visited = false;

        for(int k = i - 1; k >= 0; k--) {

            if(arr[k] == arr[i]) {
                visited = true;
                break;
            }
        }

        if(visited)
            continue;

        for(int j = i + 1; j < arr.length; j++) {

            if(arr[i] == arr[j])
                count++;
        }

        System.out.println(arr[i] + " -> " + count);
    }
}

```

25. Remove Duplicates

```

int removeDuplicates(int[] arr) {

    int j = 0;

    for(int i = 1; i < arr.length; i++) {

        if(arr[i] != arr[j]) {

            j++;
            arr[j] = arr[i];
        }
    }

    return j + 1;
}

```

26. Second Largest Element

```
int secondLargest(int[] arr) {  
  
    int first = Integer.MIN_VALUE;  
    int second = Integer.MIN_VALUE;  
  
    for(int x : arr) {  
  
        if(x > first) {  
  
            second = first;  
            first = x;  
        }  
        else if(x > second && x != first) {  
  
            second = x;  
        }  
    }  
  
    return second;  
}
```

27. Check Array Sorted

```
boolean isSorted(int[] arr) {  
  
    for(int i = 1; i < arr.length; i++) {  
  
        if(arr[i] < arr[i - 1])  
            return false;  
    }  
  
    return true;  
}
```

28. Left Rotate Array by One

```
void leftRotate(int[] arr) {  
  
    int first = arr[0];  
  
    for(int i = 0; i < arr.length - 1; i++) {  
        arr[i] = arr[i + 1];  
    }  
  
    arr[arr.length - 1] = first;  
}
```

29. Right Rotate Array by One

```
void rightRotate(int[] arr) {  
  
    int last = arr[arr.length - 1];  
  
    for(int i = arr.length - 1; i > 0; i--) {  
        arr[i] = arr[i - 1];  
    }  
  
    arr[0] = last;  
}
```

30. Kadane Algorithm

```
int maxSubarray(int[] arr) {  
  
    int maxSum = arr[0];  
    int curr = arr[0];  
  
    for(int i = 1; i < arr.length; i++) {  
        curr = Math.max(arr[i], curr + arr[i]);  
    }  
}
```

```
        maxSum = Math.max(maxSum, curr);
    }

    return maxSum;
}
```

31. Prefix Sum Array

```
int[] prefixSum(int[] arr) {
    int[] pre = new int[arr.length];

    pre[0] = arr[0];

    for(int i = 1; i < arr.length; i++) {
        pre[i] = pre[i - 1] + arr[i];
    }

    return pre;
}
```

32. Two Sum Problem

```
boolean twoSum(int[] arr, int target) {
    java.util.HashSet<Integer> set = new java.util.HashSet<>();

    for(int x : arr) {
        if(set.contains(target - x))
            return true;

        set.add(x);
    }

    return false;
}
```

33. Move Zeros to End

```
void moveZeros(int[] arr) {  
  
    int j = 0;  
  
    for(int i = 0; i < arr.length; i++) {  
  
        if(arr[i] != 0) {  
  
            int temp = arr[i];  
            arr[i] = arr[j];  
            arr[j] = temp;  
  
            j++;  
        }  
    }  
}
```

34. Merge Two Arrays

```
int[] merge(int[] a, int[] b) {  
  
    int[] ans = new int[a.length + b.length];  
  
    int k = 0;  
  
    for(int x : a)  
        ans[k++] = x;  
  
    for(int x : b)  
        ans[k++] = x;  
  
    return ans;  
}
```

35. Find Missing Number

```
int missing(int[] arr, int n) {  
  
    int sum = n * (n + 1) / 2;  
  
    int arrSum = 0;  
  
    for(int x : arr)  
        arrSum += x;  
  
    return sum - arrSum;  
}
```

36. Majority Element

```
int majority(int[] arr) {  
  
    int count = 0;  
    int candidate = 0;  
  
    for(int x : arr) {  
  
        if(count == 0)  
            candidate = x;  
  
        if(x == candidate)  
            count++;  
        else  
            count--;  
    }  
  
    return candidate;  
}
```

37. Sliding Window Maximum Sum

```
int maxWindow(int[] arr, int k) {
```

```
int sum = 0;

for(int i = 0; i < k; i++)
    sum += arr[i];

int max = sum;

for(int i = k; i < arr.length; i++) {

    sum += arr[i] - arr[i - k];

    max = Math.max(max, sum);
}

return max;
}
```

38. 2D Array Declaration

```
int[][] arr = new int[3][3];
```

39. 2D Array Traversal

```
for(int i = 0; i < arr.length; i++) {

    for(int j = 0; j < arr[i].length; j++) {

        System.out.print(arr[i][j] + " ");

    }

    System.out.println();
}
```

40. Matrix Addition

```
void addMatrix(int[][] a, int[][] b) {
```

```

int[][] c = new int[a.length][a[0].length];

for(int i = 0; i < a.length; i++) {

    for(int j = 0; j < a[0].length; j++) {

        c[i][j] = a[i][j] + b[i][j];

    }

}

```

41. Matrix Transpose

```

void transpose(int[][] arr) {

    for(int i = 0; i < arr.length; i++) {

        for(int j = i + 1; j < arr[0].length; j++) {

            int temp = arr[i][j];
            arr[i][j] = arr[j][i];
            arr[j][i] = temp;

        }

    }

}

```

42. Time Complexity

Operation	Complexity
Access	O(1)
Traversal	O(n)
Search	O(n)
Binary Search	O(log n)
Insert	O(n)
Delete	O(n)
Bubble Sort	O(n ²)

Operation	Complexity
Selection Sort	$O(n^2)$
Insertion Sort	$O(n^2)$

43. Important Interview Topics

- Kadane Algorithm
 - Two Sum
 - Prefix Sum
 - Sliding Window
 - Binary Search
 - Sorting
 - Rotation
 - Majority Element
 - Merge Arrays
 - Remove Duplicates
 - Frequency Count
 - 2D Arrays
 - Matrix Problems
 - Subarray Problems
-

44. Most Asked Interview Questions

1. Two Sum
2. Best Time to Buy & Sell Stock
3. Maximum Subarray
4. Rotate Array
5. Move Zeros
6. Merge Sorted Arrays
7. Remove Duplicates
8. Majority Element
9. Missing Number
10. Container With Most Water
11. Trapping Rain Water
12. Product of Array Except Self
13. Maximum Product Subarray
14. Next Permutation
15. Spiral Matrix